

NBchat 认证系统深度剖析

注册 & 登录全链路技术分析

一、系统概览

认证系统由三个面板组成：**登录**、**注册**、**找回密码**，承载在一个全屏分屏式页面上。左侧是插画动画区，右侧是表单区。后端由 7 个 Java 类组成从 API 入口到数据库的完整调用链。

二、前端表单系统

2.1 三面板切换

三个面板（loginPanel / registerPanel / resetPanel）本质上都在同一页面上，通过“标签页”切换，不是三个独立页面。切换时不仅替换表单，还同步更新标题和副标题。

```
// auth.js - activatePanel()
authTitle.textContent = tab.dataset.title;    // "Welcome back!" / "Create
account" / "Reset password"
authSubtitle.textContent = tab.dataset.subtitle; // 对应副标题
stage?.classList.toggle("registering", panelId === "registerPanel");
```

值得注意的细节：面板状态同步到 URL hash（`history.replaceState(null, "", "#registerPanel")`），用户可以收藏注册页链接直接进入对应面板。

2.2 QQ 邮箱前端校验

邮箱输入框做了两层校验。第一层是 HTML5 `type="email"` 的原生校验。第二层是 JavaScript 正则：

```
function isQqEmail(value) {
    return /^[1-9][0-9]{4,11}@qq\.com$/i.test(String(value || "").trim());
}
```

后端 `Validation.java` 中用的是**完全相同的正则** `^[1-9][0-9]{4,11}@qq\\.com$`。前后端双校验防止用户在 HTML5 校验被绕过的情况下提交非法邮箱。

2.3 验证码发送按钮的冷却机制

这是前端最精妙的设计之一。发送验证码不是无限制的——每次发送后有 180 秒冷却。

冷却状态存储在 `localStorage` 中，key 的格式为 `codeCooldownUntil:{register|reset}:{邮箱地址}`。这样即使用户刷新页面、关闭浏览器，冷却倒计时也不会重置。

```
// 冷却计算
localStorage.setItem(key, String(Date.now() + 180 * 1000)); // 存储过期时间戳
remaining = Math.ceil((过期时间 - 当前时间) / 1000); // 计算剩余秒数
```

按钮在冷却期间显示倒计时数字（如"156s"）并处于 disabled 状态。每秒更新一次。后端 `EmailCodeCooldown.java` 中也有一个完全独立的 180 秒冷却——即使前端冷却被绕过，后端也会拒绝。

2.4 密码可见性切换

密码输入框右侧有一个  按钮。点击后 `input.type` 在 "password" 和 "text" 之间切换，按钮文字在  和  之间变化。`aria-label` 同步更新为"显示密码"或"隐藏密码"——这是无障碍设计的基本要求。

焦点管理：切换后 `input.focus()` 让光标回到密码框，不中断用户输入。

三、插画动画系统

登录页左侧的角色插画是一个纯 CSS + JS 实现的交互式动画系统。

3.1 眼球追踪

`document.addEventListener("pointermove")` 监听全局鼠标移动。每个眼球的瞳孔是一个 `<span>` 元素，通过计算鼠标位置与眼球中心的距离和角度，用 `transform: translate()` 将瞳孔移向鼠标方向。

```
const angle = Math.atan2(mouseY - eyeCenterY, mouseX - eyeCenterX);
const distance = Math.hypot(mouseX - eyeCenterX, mouseY - eyeCenterY);
const maxDistance = 9;
const ratio = Math.min(1, distance / 180);
const easedDistance = maxDistance * (1 - Math.pow(1 - ratio, 2)); // 缓出效果
```

关键是 `ratio * (1 - ratio)` 的缓出公式——距离越近，瞳孔响应越灵敏；距离越远，响应越迟钝。这模拟了真实眼球运动的非线性特性。

3.2 密码输入时的隐私模式

输入密码时（`type="password" + 有内容`），角色卡牌集体向远离表单的方向倾斜（`password-secret` class），瞳孔统一看向别处。这是一种拟人化的隐私尊重的视觉表达。

3.3 密码可见时的偷看模式

当用户点击  让密码明文显示时，角色进入 `password-peeking` 状态。系统启动一个定时器循环：等待 2~5 秒随机间隔 → 卡牌短暂地探向表单方向偷看（800ms）→ 退回 → 再次等待。

```
authState.passwordPeekTimer = setTimeout(() => {
  stage.classList.add("password-peeking");
  authState.passwordPeekReturnTimer = setTimeout(() => {
    stage.classList.remove("password-peeking");
  }, 800);
}, 2000 + Math.random() * 1000);
```

```
        schedulePasswordPeekLoop(); // 递归调度下一次
    }, 800);
}, Math.random() * 3000 + 2000);
```

3.4 注册时的兴奋模式

切换到注册面板时，`stage.classList.add("registering")` 触发卡牌更大幅度的倾斜动画。

3.5 随机眨眼

每个角色绑定了一个递归的 `scheduleBlink()` 函数。每次间隔 3~7 秒随机时间，卡牌短暂添加 `blinking` class（150ms，瞳孔缩小消失），然后移除 class 并递归调度下一次。

四、后端认证流程

4.1 API 入口 — AuthResource

`AuthResource` 是 8 个 REST 端点的集合：

端点	方法	用途
<code>/auth/login</code>	POST	登录
<code>/auth/register</code>	POST	注册
<code>/auth/register/code</code>	POST	发送注册验证码
<code>/auth/password-reset</code>	POST	重置密码
<code>/auth/password-reset/code</code>	POST	发送重置验证码
<code>/auth/logout</code>	POST	登出
<code>/auth/me</code>	GET	检查登录状态

4.2 频率限制

`AuthResource` 在类级别定义了两个静态 `RateLimiter`：

- 登录限流：每 5 分钟 8 次 (`new RateLimiter(8, Duration.ofMinutes(5))`)
- 验证码限流：每 10 分钟 3 次

限流的 key 是 `IP地址:用户名`（或 `IP地址:邮箱`），这意味着同一 IP 对同一账号的频繁请求会被拦截，但换一个用户名可以继续尝试——这在安全和可用性之间取得了平衡。

IP 地址的提取考虑了反向代理场景：优先读 `X-Forwarded-For` 头（Nginx 设置），fallback 到 `request.getRemoteAddr()`。

4.3 登录流程

```

前端 → POST /auth/login {username, password}
      ↓
AuthResource.login()
      ↓ 调用 requireRateLimit() 检查频率限制
      ↓
AuthService.login()
      ↓ 调用 Validation.notBlank() 校验用户名和密码非空
      ↓ 调用 authDao.passwordHashByUsername() 查询密码哈希
      ↓ 调用 PasswordHasher.verify() 比对密码
      ↓ 调用 authDao.findByUsername() 查询完整用户信息
      ↓
AuthResource 设置 HttpSession.USER_ID = user.id()
      ↓
返回 ApiResponse<User>

```

时序攻击防护：登录流程先查密码哈希，再用 BCrypt 比对，而不是先判断用户名是否存在。即使用户不存在，`passwordHashByUsername()` 返回 null，`PasswordHasher.verify()` 仍然执行完整的哈希比对过程（返回 false），耗时与成功验证相近。这让攻击者无法通过响应时间差异来枚举有效用户名。

BCrypt 参数：`BCrypt.gensalt(12)` 定义了 $2^{12} = 4096$ 轮的哈希计算量。每次密码验证大约消耗 200~300ms，这对用户体验无感知，但对暴力攻击者来说意味着每秒只能尝试 3~5 个密码。

4.4 注册流程

```

前端 → POST /auth/register {username, password, qqEmail, code, nickname}
      ↓
AuthResource.register()
      ↓
AuthService.register()
      ↓ 调用 normalize() 统一 trim 输入 + normalizeQqEmail 小写化邮箱
      ↓ 调用 validateRegister() 校验：
      |   - 用户名 ≥ 3 字符
      |   - 密码 ≥ 6 字符
      |   - QQ 邮箱格式合法（正则 ^[1-9][0-9]{4,11}@qq\.com$）
      |   - 验证码非空
      ↓ 调用 authDao.usernameExists() + authDao.emailExists() 检查重复
      ↓ 调用 authDao.consumeEmailCode() 校验验证码
      ↓ 调用 PasswordHasher.hash() 生成 BCrypt 哈希
      ↓ 调用 authDao.createUser() 插入用户 + 自动创建默认好友分组
      ↓
AuthResource 将新用户的 ID 写入 HttpSession（注册即登录）
      ↓
返回 ApiResponse<User>

```

用户名重复防护：`usernameExists()` 查询 `SELECT 1 FROM users WHERE username=?`，利用数据库 UNIQUE 约束在 SQL 层面防止竞态条件。

邮箱重复防护：同理。值得注意的是，QQ 邮箱在存储和比较前都被 `normalizeQqEmail()` 统一转为小写，防止 `User@QQ.com` 和 `user@qq.com` 被视为不同邮箱。

昵称 fallback：如果用户没有输入昵称，昵称默认为用户名。

4.5 验证码发送流程

```

前端 → POST /auth/register/code {qqEmail}
      ↓
AuthResource.sendRegisterCode()
      ↓ 调用 requireRateLimit() (10分钟3次)
      ↓
AuthService.sendRegisterCode()
      ↓ 调用 Validation.normalizeQqEmail() 统一小写化
      ↓ 查询数据库—如果邮箱已注册，拒绝发送
      ↓
EmailCodeService.send()
      ↓ 调用 EmailCodeCooldown.remainingSeconds() 检查 180 秒冷却
      ↓ 调用 authDao.markExistingCodesUsed() 将之前的验证码标记为已使用
      ↓ 调用 SecureRandom.nextInt(1_000_000) 生成 6 位随机码
      ↓ 调用 authDao.saveEmailCode() 存入数据库 (过期时间 = now + 10 分钟)
      ↓ 调用 QqMailSender.sendCode() 发送邮件
  
```

冷却的双重保障：`EmailCodeCooldown.remainingSeconds()` 在服务端检查上一个验证码的发送时间，不足 180 秒直接拒绝。`markExistingCodesUsed()` 确保同一邮箱同一用途之前的所有未使用验证码全部失效——防止攻击者用同一邮箱反复请求验证码然后猜测试用旧码。

邮件内容：

你正在使用 Jakarta Chat 注册账号。

验证码：482931

有效期：10 分钟。

如果不是你本人操作，请忽略这封邮件。

开发模式下 (`mail.devMode=true`)，`QqMailSender` 不发送真实邮件，而是用 `System.out.printf()` 打印到控制台。这个设计让本地开发不需要配置 SMTP。

4.6 验证码校验

`consumeEmailCode()` 的 UPDATE 语句包含了五个条件：

```

UPDATE email_verification_codes SET used=1
WHERE qq_email=?      -- 邮箱匹配
  AND code=?          -- 验证码匹配
  AND purpose=?       -- 用途匹配 (REGISTER vs RESET_PASSWORD)
  AND used=0          -- 未被使用过
  
```

```
AND expires_at>NOW() -- 未过期
AND id=(SELECT MAX(id) FROM ... WHERE qq_email=? AND purpose=?)
```

最后一个子查询保证只校验“最新的那条验证码”——历史的验证码即使未过期也不会被接受。`Jdbc.update()` 返回受影响行数，如果为 0 则返回“验证码无效或已过期”。

4.7 找回密码流程

找回密码与注册类似，但有额外的安全检查：

1. 发送验证码前检查 `emailExists()`——只给已注册的邮箱发送，防止验证码被滥用到不存在的账号
2. 重置密码时再次检查 `emailExists()`
3. 密码 hash 通过 `updatePassword()` 直接 UPDATE 用户记录

4.8 登出

登出简单直接——调用 `HttpSession.invalidate()` 销毁整个 Session。Tomcat 会移除该 Session 的所有属性并回收内存。`getSession(false)` 的 `false` 参数表示不创建新 Session。

五、从浏览器到数据库的完整数据流

以一次注册为例：

浏览器

用户在输入框中打字
 @ 图标表示用户名输入框
 ✉ 图标表示邮箱输入框
 □ 图标表示密码输入框 + ☉ 切换可见性
 # 图标表示验证码输入框 + [发送] 按钮（180s 冷却）

POST /api/auth/register
 Content-Type: application/json
 Body:

```
{"username":"alice","password":"123456","qqEmail":"10001@qq.com","code":"482931","nickname":"Alice"}
```

▼ Nginx (如果配置了反向代理)

```
proxy_pass http://127.0.0.1:8080/v1_2026_5_30/
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for
```

▼ Tomcat 10.1

Jersey Servlet 根据 `@ApplicationPath("/api")` 路由请求

▼ AuthResource.register()

HttpServletRequest → 获取 Session → 获取 RemoteAddr

```

AuthService.register()
| 1. 规范化输入 (trim + 小写化邮箱)
| 2. 校验字段
| 3. 查询用户名/邮箱是否已存在
| 4. 消费验证码
| 5. BCrypt 哈希密码
| 6. 创建用???记录
| 7. 自动创建好友分组 "My Friends"
| 8. 返回 User record
|
▼
AuthResource.register()
| HttpSession.setAttribute("CHAT_USER_ID", user.id())
| 返回 JSON: {"ok":true,"message":"ok","data":{...}}
|
▼
浏览器
| auth.js 收到响应
| location.href = "app.html" 跳转到主页面

```

六、安全设计清单

措施	位置	防护目标
BCrypt 12 rounds	PasswordHasher	密码泄露后不可逆
频率限制（登录）	AuthResource	暴力破解
频率限制（验证码）	AuthResource	短信轰炸
验证码 180s 冷却（前端）	auth.js + localStorage	用户重复点击
验证码 180s 冷却（后端）	EmailCodeCooldown	绕过前端限制
验证码 10 分钟过期	schema.sql + SQL WHERE	验证码长期有效
旧码自动失效	markExistingCodesUsed()	验证码复用攻击
QQ 邮箱正则校验（前端）	isQqEmail()	格式错误
QQ 邮箱正则校验（后端）	Validation.qqEmail()	绕过前端校验
邮箱小写化存储	normalizeQqEmail()	大小写混淆
注册即登录	HttpSession.setAttribute()	注册后重新登录
登出销毁 Session	HttpSession.invalidate()	Session 残留
XSS 防护	escapeHtml() 前端渲染	消息内容注入脚本